



Python Programming Foundations

Week 4 - Debugging, Testing, & Reading Structured Code
Day 2: "Reading Procedural Function and Class Code"

1



Different Code Shapes Can Do The Same Job

The same task can be written in different shapes.

Today, compare:

- ▶ procedural code
- ▶ function-based code
- ▶ class-based code

The goal is not to decide that one shape is always best. The goal is to read the structure and explain how it works.

2



Same Task, Three Ways

Task: Track tasks — add a task, list tasks, mark a task as done.

1. PROCEDURAL

Use data and functions that operate on it.

DATA (tasks list)

- 0 "Buy milk"
- 1 "Study Python"
- 2 "Walk the dog"

FUNCTIONS

- add_task(tasks, text)
- list_tasks(tasks)
- mark_done(tasks, index)

Example Use

```
tasks = []
add_task(tasks, "Buy milk")
add_task(tasks, "Study Python")
mark_done(tasks, 1)
list_tasks(tasks)
```

2. FUNCTION-BASED

Use functions that hide the details.

ABSTRACT VIEW

TASK MANAGER

- add_task(text)
- list_tasks()
- mark_done(index)

Example Use

```
tm = create_task_manager()
tm.add_task("Buy milk")
tm.add_task("Study Python")
tm.mark_done(1)
tm.list_tasks()
```

3. CLASS-BASED

Use a class to group data and behavior.

OBJECT VIEW

TaskManager

Data (inside each object)

- tasks (list)

Methods

- add_task(text)
- list_tasks()
- mark_done(index)

Example Use

```
tm = TaskManager()
tm.add_task("Buy milk")
tm.add_task("Study Python")
tm.mark_done(1)
tm.list_tasks()
```

Same Task, Three Styles

3



Build Structure, Then Read Structure

Week 3 focused on building structure.

Week 4 adds code literacy: reading structure someone else wrote.

Code reading matters because:

- ▶ examples may use unfamiliar organization
- ▶ AI may generate class-based code
- ▶ documentation may assume structure vocabulary
- ▶ debugging depends on knowing where to look

4



What Counts as Success Today

Read the structure -> identify the parts -> explain in plain language -> modify one small thing.

For structured code, ask:

- What stores data?
- What performs actions?
- Where does the program start?
- What line changes behavior?

5



A Simple Reading Path

Understand first. Then change one small thing with confidence.

- 1 READ THE STRUCTURE**
Get the big picture. What does this do?

```
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print(average)
```


What is the overall flow from start to finish?
- 2 IDENTIFY THE PARTS**
Name the important pieces. What is each part for?

```
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print(average)
```


Can you label each part with its job?
- 3 EXPLAIN IN PLAIN LANGUAGE**
Say what it does out loud. Like you're teaching it.
Start with a total of 0.
Loop at each score in the list.
Add the score to the running total.
Find the average by dividing the total by how many scores there are.
Print the average.
Could someone else understand your explanation?
- 4 MODIFY ONE SMALL THING**
Make one tiny change. Then run it and see.
Example small change: Round the average to 2 decimals.

```
average = total / len(scores)
average = round(average, 2)
```


Did it work? What happened?

The Goal: Read with purpose. Understand deeply. Change with confidence. Remember: Small steps lead to strong understanding.

Today's Success

6

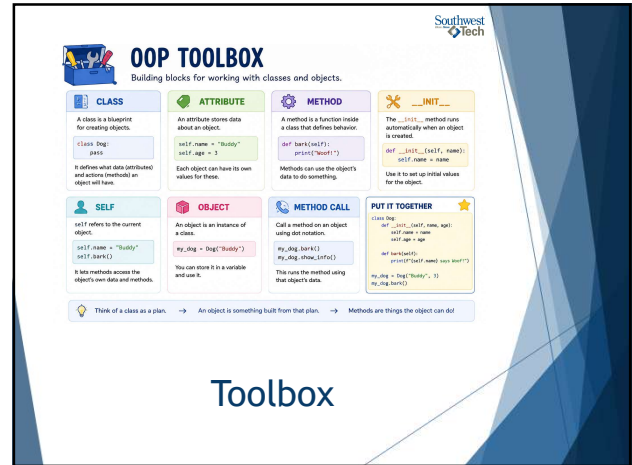


What We Will Use Today

- ▶ procedural code
- ▶ functions
- ▶ classes
- ▶ attributes
- ▶ methods
- ▶ `__init__`
- ▶ `'self'`
- ▶ object creation

These are recognition tools for reading small structured examples.

7



OOP TOOLBOX

Building blocks for working with classes and objects.

- CLASS**: A class is a blueprint for creating objects.


```
class Dog:
    pass
```
- ATTRIBUTE**: An attribute stores data about an object.


```
self.name = "Buddy"
self.age = 3
```
- METHOD**: A method is a function inside a class that defines behavior.


```
def bark(self):
    print("woof!")
```
- __INIT__**: The `__init__` method runs automatically when an object is created.


```
def __init__(self, name):
    self.name = name
```
- SELF**: `self` refers to the current object.


```
self.name = "Buddy"
self.bark()
```
- OBJECT**: An object is an instance of a class.

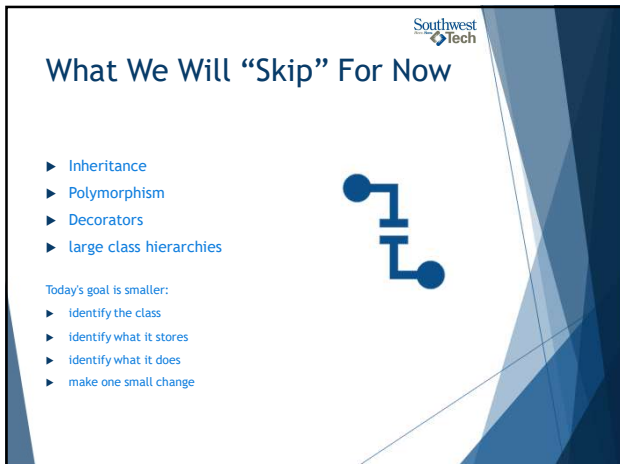

```
my_dog = Dog("Buddy")
```
- METHOD CALL**: Call a method on an object using dot notation.


```
my_dog.bark()
```
- PUT IT TOGETHER**:


```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        print(f"{self.name} says woof!")
```

Think of a class as a plan → An object is something built from that plan → Methods are things the object can do

8



What We Will "Skip" For Now

- ▶ Inheritance
- ▶ Polymorphism
- ▶ Decorators
- ▶ large class hierarchies

Today's goal is smaller:

- ▶ identify the class
- ▶ identify what it stores
- ▶ identify what it does
- ▶ make one small change

9



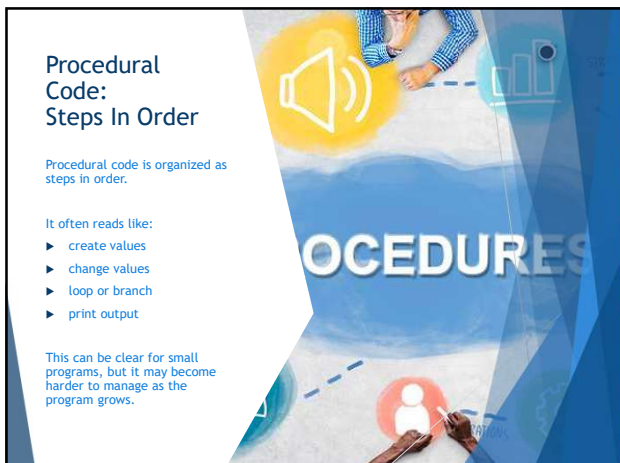
Parked For Later

Useful later, not today's focus.

- INHERITANCE**: One class can inherit attributes and methods from another class.
- POLYMORPHISM**: Different objects can respond to the same method call in their own way.
- DECORATORS**: A way to add extra behavior to functions or methods.
- LARGE CLASS HIERARCHIES**: Designing and managing many related classes in a bigger structure.

We'll use these ideas soon. For now, let's build a strong foundation.

10



Procedural Code: Steps In Order

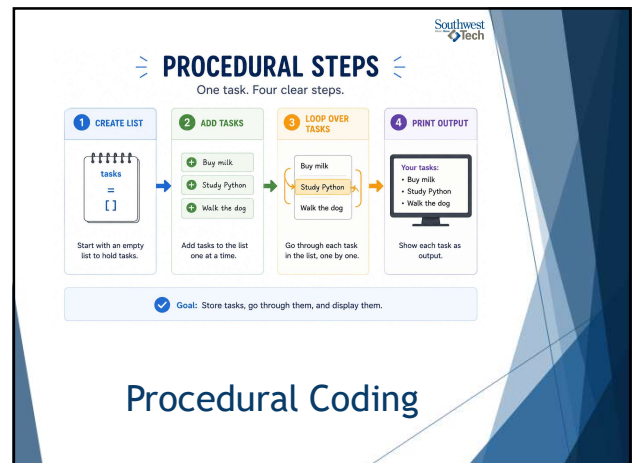
Procedural code is organized as steps in order.

It often reads like:

- ▶ create values
- ▶ change values
- ▶ loop or branch
- ▶ print output

This can be clear for small programs, but it may become harder to manage as the program grows.

11



PROCEDURAL STEPS

One task. Four clear steps.

- 1 CREATE LIST**: Start with an empty list to hold tasks.


```
tasks = []
```
- 2 ADD TASKS**: Add tasks to the list one at a time.


```
tasks.append("Buy milk")
tasks.append("Study Python")
tasks.append("Walk the dog")
```
- 3 LOOP OVER TASKS**: Go through each task in the list, one by one.


```
for task in tasks:
    print(task)
```
- 4 PRINT OUTPUT**: Show each task as output.


```
print(tasks)
```

Goal: Store tasks, go through them, and display them.

12



13



14



15



16



17



18

Discount Calculation
Item price: \$100
Discount: 20%

Expected
\$80.00
100 - 20% of 100 = 80

Actual
\$120.00
This doesn't look right.

Let's find where the calculation went wrong.

Calculation Steps

- Start with item price: 100.00
- Calculate discount (20% of 100): 20.00
- Apply discount to price: $price = price - discount$: 120.00
- Tax (0%): 0.00
- Final price: 120.00

The discount should be subtracted, not added. $price = price - discount$

**Demo 2:
Function Based Task Tracker**

19

Class Based Coding

This Photo by Unknown Author is licensed under CC BY-SA-NC

20

A Class Stores Data And Actions Together

A class can group data and actions that belong together.

Beginning terms:

- **attribute**: data the object stores
- **method**: action the object can do
- **object**: one usable instance of the class

This is organization, not magic.

21

TaskTracker
A simple container that stores tasks and provides actions to work with them.

DATA
tasks (a list of task strings)
• Buy milk
• Study Python
• Walk the dog
This holds the tasks. It can grow as we add more.

ACTIONS
• `add_task(task)`
Adds a new task to the tasks list.
• `show_tasks()`
Shows each task in the tasks list.

TaskTracker keeps the data (tasks) and knows what actions we can do.

Example flow:
1. `ADD A TASK`
`tracker.add_task("Read a book")`
2. `SHOW TASKS`
`tracker.show_tasks()`
3. **OUTPUT**
• Buy milk
• Study Python
• Walk the dog
• Read a book

Class: Data & Actions

22

`__init__` Sets The Starting State

`__init__` is the setup step for a new object. It often creates starting attributes.

Example idea:
`new_tracker -> setup -> tracker has an empty task list`

Say "setup step" before worrying about the word constructor.

23

HOW A NEW TRACKER STARTS
Simple flow: create → setup → ready

- 1 CREATE NEW TRACKER**
We ask for a new TaskTracker object.
A brand new tracker is created.
- 2 `__INIT__` SETUP**
The `__init__` method runs automatically to set things up.
This prepares the tracker with starting values.
- 3 EMPTY TASK LIST STORED**
The tracker now has an empty list ready to use.
tasks = []
Ready to add tasks whenever we want.

Key idea: Every new tracker runs its setup step (`__init__`) and starts with an empty task list.

Class Start Up

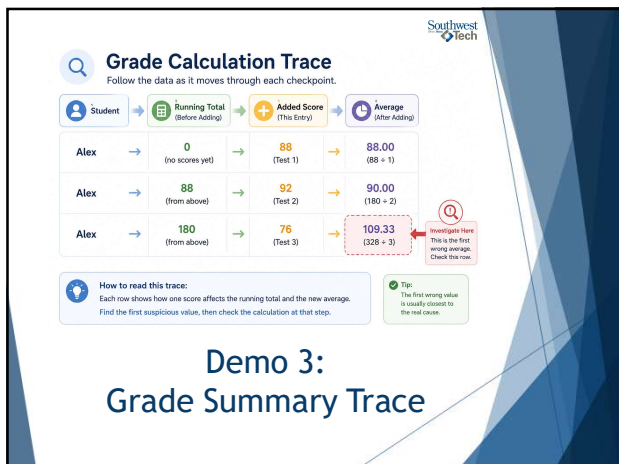
24



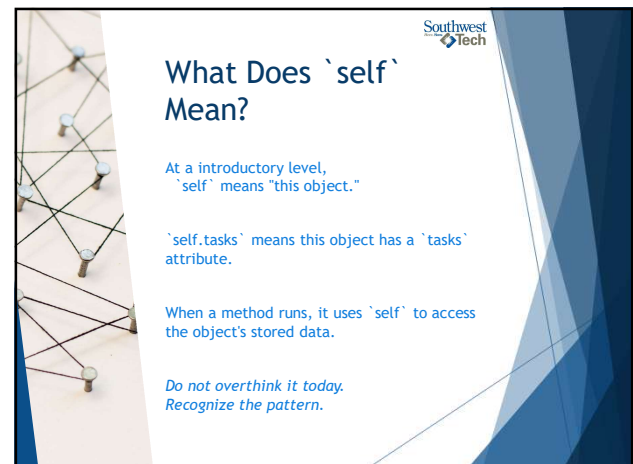
25



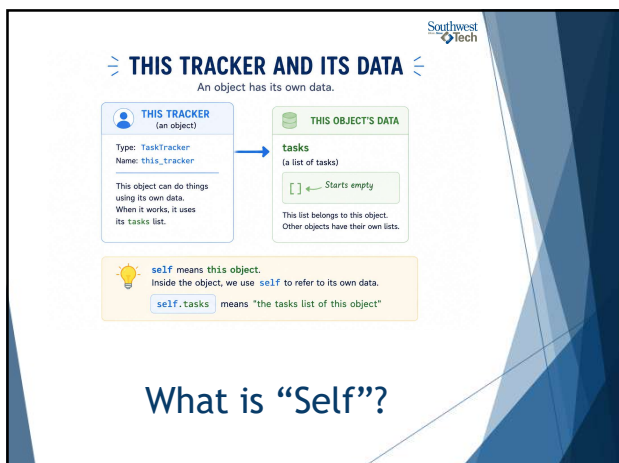
26



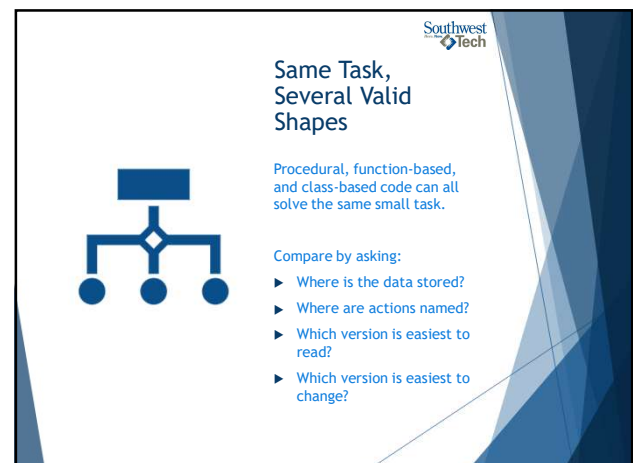
27



28



29



30

Southwest
Tech

Common Failure: Prestige Bias

Class-based code is not automatically better.

Choose or evaluate structure by fit:

- ▶ Does the structure match the problem?
- ▶ Can you explain it?
- ▶ Can you modify it safely?
- ▶ Is it small enough for this course target?

Advanced-looking code is not useful if you cannot understand it.

31

Southwest
Tech

CHOOSE WHAT FITS THE PROBLEM

Good design is about clarity and flexibility, not complexity.

ADVANCED-LOOKING
Uses more structure than needed for this simple task.

```
class TaskManager:
    def __init__(self):
        self.tasks = []
        self._listeners = []
    def add_task(self, task):
        self._validate_task(task)
        self._tasks.append(task)
        self._log_change()
    def show_tasks(self):
        for i in self.tasks:
            print(f"{i}")
# ... more helper methods ...
```

More parts to learn
More to maintain and test
Harder to explain at a glance

THE GOAL
Track a small set of tasks in memory and show them.

FITS THE PROBLEM
Simple, direct, and easy to work with for what we need right now.

```
tasks = []
def add_task(task):
    tasks.append(task)
def show_tasks():
    for i in tasks:
        print(f"{i}")
```

Clear and easy to explain
Anyone can use what it does.
Simple to modify
Change or add features quickly.
Just enough structure
Matches the size of the problem.

CHOOSE FIT-FOR-PURPOSE
Because it is:

- ✓ Explains
- ✓ Easy to read and understand.
- ✓ Modifiable
- ✓ Easy to change and extend.

GOOD RULE: Start simple. Add structure only when the problem grows.
The best code is the code that is easy to understand and easy to change.

Common Failure: Prestige Bias

32

Southwest
Tech

Common AI Failure!

In AI Generated code:

The AI may return class-based solutions that *look polished* but exceed the current need.

33

Southwest
Tech

Assignment #7

For Assignment 7, read and lightly modify a structured example.

Your work should show that you can:

- ▶ identify the class
- ▶ identify attributes
- ▶ identify methods
- ▶ create or use an object
- ▶ call a method
- ▶ make one small working change

34

Southwest
Tech

Structured Code Reading Checklist

Before editing class-based code, answer:

- ▶ What is the class name?
- ▶ What does the object store?
- ▶ What actions can it do?
- ▶ Where is the object created?
- ▶ Where are methods called?
- ▶ What one change am I making?

35

Southwest
Tech

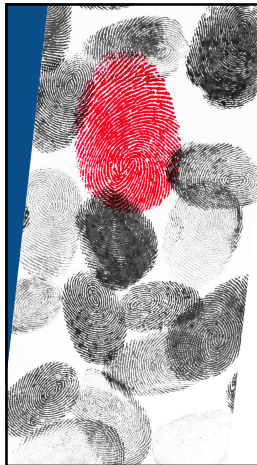
Structure Code Checklist

MY CLASS CHECKLIST
Did I build it, use it, and make a change?

- ✓ 1 class name
- ✓ 2 stored data
- ✓ 3 actions
- ✓ 4 object creation
- ✓ 5 method calls
- ✓ 6 one change

Small steps. Clear code. Real learning.

36



Southwest
Tech

Assignment #7 Evidence

Useful evidence includes:

- ▶ modified `.py` file
- ▶ short explanation of the class
- ▶ attributes identified
- ▶ methods identified
- ▶ one working modification
- ▶ comparison to procedural or function-based structure

Explain the code in your own words.

37



Southwest
Tech

Evidence of Understanding

Class: TaskTracker

MODIFIED FILE
task_tracker.py

```
class TaskTracker:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append(task)

    def show_tasks(self):
        if not self.tasks:
            print("No tasks yet.")
        else:
            print(f"Your tasks are:")
            for i, task in enumerate(self.tasks, 1):
                print(f"{i}. {task}")

    # Small change I made:
    def count_tasks(self):
        return len(self.tasks)
```

OUTPUT SAMPLE (from running my program)

```
Your tasks:
1. Study Python
2. Read chapter 3
3. Complete lab
```

CLASS EXPLANATION

TaskTracker keeps a list of tasks, I added tasks, can show them, and can count how many there are.

Change I made:
Added count_tasks() to return the number of tasks.

MY CHANGE
I added count_tasks() so the program can tell me how many tasks I have. This helps me see the data in a new way.

ATTRIBUTES & METHODS

ATTRIBUTE (stored data)
self.tasks → list of tasks

METHODS (actions)
...init...() → Start up the object with an empty list.
add_task(self, task) → Add a new task to the list.
show_tasks(self) → Print all tasks (or a message if empty).
count_tasks(self) → Returns how many tasks are stored.

WHAT THIS SHOWS

- ✓ I built a working class.
- ✓ I documented its data and actions.
- ✓ I can make a change to improve it.

38



Southwest
Tech



Thank you!

Have Fun!

39